Pandas Python

Complete Reference Notes with Theory & Examples

Covers Series · DataFrame · I/O · Selection · Cleaning · GroupBy · Merge · Apply · DateTime · Pivot · Stats & more

Table of Contents

01 Installation & Import	12 Merging & Joining
02 Series – 1-D Labelled Array	13 Concatenating
03 DataFrame – 2-D Table	14 Apply, Map & ApplyMap
04 Reading & Writing Data (I/O)	15 String Operations
05 Viewing & Inspecting Data	16 DateTime Operations
06 Selecting Data (loc / iloc / Boolean)	17 Pivot Tables & Reshaping
07 Adding & Modifying Data	18 Value Counts & Unique
08 Dropping Data	19 Index Operations
09 Handling Missing Values	20 Statistical Functions
10 Sorting	21 Conditional Selection
11 GroupBy & Aggregation	22 Quick Reference Cheat Sheet

SECTION 01

Installation & Import

Pandas is an open-source Python library built on top of NumPy, designed for fast, flexible data manipulation and analysis. Before using it, you must install it (once) with pip and then import it into every script or notebook. The conventional alias 'pd' is universally adopted so that every Pandas call is prefixed with pd.

■ Installation & Import

```
# Install (run once in terminal)
pip install pandas numpy


# Import in every script/notebook
import pandas as pd
import numpy as np


# Verify version
print(pd.__version__) # e.g. 2.2.0
```

SECTION 02

Series – 1-D Labelled Array

A Series is the fundamental one-dimensional data structure in Pandas. Think of it as a single column of a spreadsheet. Every element has an associated label called an index. By default the index is 0, 1, 2 … but you can assign any custom labels (strings, dates, etc.). A Series can hold integers, floats, strings, booleans, or Python objects. You can create a Series from a Python list, a NumPy array, a scalar value, or a dictionary.

■ Creating a Series

```
# From a plain list (auto index 0,1,2...)
s = pd.Series([10, 20, 30, 40])
print(s)
# 0 10
# 1 20 dtype: int64


# With custom string index
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s['b']) # 20


# From a dictionary (keys become index)
```

```
s = pd.Series({'x': 100, 'y': 200, 'z': 300})
print(s['z']) # 300


# Scalar broadcast
s = pd.Series(5, index=['a','b','c']) # all values = 5
```

■ *Series supports vectorised arithmetic: pd.Series([1,2,3]) * 2 → [2,4,6]*

SECTION 03

DataFrame – 2-D Table

A DataFrame is a two-dimensional, size-mutable, and potentially heterogeneous tabular data structure — similar to a SQL table or an Excel spreadsheet. Each column is a Series, and all columns share the same index (row labels). You can create a DataFrame from a dictionary of lists, a list of dictionaries, a 2-D NumPy array, another DataFrame, or by reading a file. Columns can have different data types (int, float, str, bool, datetime).

■ Creating a DataFrame

```
# From dictionary of lists (most common)
df = pd.DataFrame({
'Name': ['Alice', 'Bob', 'Charlie'],
'Age': [25, 30, 35],
'City': ['Delhi', 'Mumbai', 'Kolkata']
})
print(df)


# From list of dictionaries
df = pd.DataFrame([
{'Name': 'Alice', 'Age': 25},
{'Name': 'Bob', 'Age': 30}
])


# From 2-D list with explicit column names
df = pd.DataFrame([[1,2],[3,4]], columns=['A','B'])


# Specify index
df = pd.DataFrame({'Score':[90,85]}, index=['Alice','Bob'])
```

SECTION 04

Reading & Writing Data (I/O)

Pandas provides a rich set of I/O functions that let you load data from—and save data to—almost any common format. The read_* family of functions returns a DataFrame. The to_* methods export a DataFrame. Setting index=False in to_csv / to_excel prevents Pandas from writing the row-number index as an extra column.

■ I/O Examples

```
# ── CSV ──────────────────────────────────────────
df = pd.read_csv('data.csv') # read
df = pd.read_csv('data.csv', sep=';') # semicolon-separated
df.to_csv('output.csv', index=False) # write (no row numbers)

# ── Excel ────────────────────────────────────────
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
df.to_excel('output.xlsx', index=False)

# ── JSON ─────────────────────────────────────────
df = pd.read_json('data.json')
df.to_json('output.json', orient='records')

# ── SQL ──────────────────────────────────────────
import sqlite3
conn = sqlite3.connect('database.db')
df = pd.read_sql('SELECT * FROM users', conn)
df.to_sql('users_copy', conn, if_exists='replace', index=False)
```

■ *Use dtype={'col': str} in read_csv to prevent Pandas from converting e.g. phone numbers to int.*

SECTION 05

Viewing & Inspecting Data

Before analysing a dataset you should always inspect it first. Pandas provides several quick-look methods: head() and tail() show a few rows, describe() gives descriptive statistics, and info() summarises dtypes, non-null counts, and memory usage. These are the first commands you should run on any new dataset.

■ Inspecting a DataFrame

```
df.head() # first 5 rows (default)
df.head(10) # first 10 rows
df.tail(3) # last 3 rows
```

```
df.shape # (rows, columns) e.g. (150, 5)
df.dtypes # data type of every column
df.info() # non-null counts + dtypes + memory
df.describe() # count, mean, std, min, 25%, 50%, 75%, max
df.describe(include='all') # include string/object cols too

df.columns # Index of column names
df.index # Row index (RangeIndex or custom)
df.size # total elements = rows * cols
df.ndim # 2 for DataFrame, 1 for Series
df.memory_usage() # bytes used per column
```

SECTION 06

Selecting Data (loc / iloc / Boolean)

There are three primary ways to select data: (1) column/bracket indexing for columns, (2) .loc[] for label-based selection of rows and columns by their actual names, and (3) .iloc[] for purely positional (integer-based) selection — like indexing a NumPy array. Boolean indexing lets you filter rows by applying conditions that return a True/False mask. Combine multiple conditions with & (AND) and | (OR) — each condition must be in parentheses.

■ Selection Examples

```
# Column selection
df['Name'] # single column → Series
df[['Name', 'Age']] # multiple columns → DataFrame

# .loc – label based
df.loc[0] # row with label 0
df.loc[0, 'Name'] # row 0, column 'Name'
df.loc[0:2, 'Name':'Age'] # row slice, col slice (inclusive)

# .iloc – integer position based
df.iloc[0] # first row
df.iloc[-1] # last row
df.iloc[0, 1] # row 0, column index 1
df.iloc[0:3, 0:2] # first 3 rows, first 2 cols
```

```
# Boolean / conditional filtering
df[df['Age'] > 25]
df[(df['Age'] > 25) & (df['City'] == 'Mumbai')]
df[df['Name'].isin(['Alice', 'Bob'])]
```

■ *loc includes the end label in slicing; iloc excludes it — same as Python list slicing.*

SECTION 07

Adding & Modifying Data

You can add new columns simply by assigning to df['new_col']. Modifying an existing column works the same way — the assignment overwrites the column in place. The rename() method renames columns or index labels using a mapping dictionary. astype() converts a column's dtype (e.g., object → int). Using inplace=True avoids needing to reassign the result.

■ Adding & Modifying Columns

```
# Add a new column
df['Salary'] = [50000, 60000, 70000]

# Modify an existing column (vectorised)
df['Age'] = df['Age'] + 1

# Derived column using apply + lambda
df['Senior'] = df['Age'].apply(lambda x: True if x >= 30 else False)

# Rename columns
df.rename(columns={'Name': 'Employee', 'Age': 'Years'}, inplace=True)

# Change dtype
df['Age'] = df['Age'].astype(float)
df['Score'] = df['Score'].astype(int)

# Insert at specific position
df.insert(1, 'Department', ['HR','IT','Finance'])
```

SECTION 08

Dropping Data

drop() removes rows or columns by label. Use axis=1 to drop columns and axis=0 (default) to drop rows. Pass inplace=True to modify the DataFrame directly rather than returning a new one. drop_duplicates() removes rows where all (or selected) values are repeated, keeping the first occurrence by default.

■ Dropping Rows & Columns

```
# Drop a single column
df.drop('City', axis=1, inplace=True)


# Drop multiple columns
df.drop(['City', 'Salary'], axis=1, inplace=True)


# Drop rows by index label
df.drop([0, 2], axis=0, inplace=True)


# Drop all duplicates (keep first by default)
df.drop_duplicates(inplace=True)


# Drop duplicates based on specific columns
df.drop_duplicates(subset=['Name', 'City'], inplace=True)


# Drop rows where all values are NaN
df.dropna(how='all', inplace=True)
```

SECTION 09

Handling Missing Values

Real-world data almost always contains missing values, represented as NaN (Not a Number) in Pandas. isnull() returns a boolean mask; summing it gives the count of missing values per column. You can handle NaN either by dropping affected rows/columns (dropna) or by filling them with a sensible value (fillna). Forward fill (ffill) propagates the last valid value forward; back fill (bfill) propagates the next valid value backward.

■ Missing Value Handling

```
# Detect missing values
df.isnull() # True where NaN
df.isnull().sum() # count per column
df.isnull().sum().sum() # grand total of NaNs


# Drop rows / columns with NaN
```

```
df.dropna() # drop rows with any NaN

df.dropna(axis=1) # drop columns with any NaN

df.dropna(subset=['Age','City']) # only check these cols

df.dropna(thresh=3) # keep rows with >= 3 non-NaN


# Fill NaN

df.fillna(0) # fill all NaN with 0

df['Age'].fillna(df['Age'].mean()) # fill with column mean

df.fillna(method='ffill') # forward fill

df.fillna(method='bfill') # backward fill

df.fillna({'Age':0,'City':'Unknown'}) # per-column fill values
```

■ *After fillna, always verify with df.isnull().sum() that no NaNs remain.*

SECTION 10

Sorting

sort_values() orders rows by the values in one or more columns. ascending=True (default) gives ascending order; False gives descending. When sorting by multiple columns, pass a list of column names and a matching list of ascending booleans. sort_index() sorts by the row index (useful after filtering which scrambles index order). na_position controls where NaN values appear: 'last' (default) or 'first'.

■ Sorting Examples

```
# Sort by a single column ascending

df.sort_values('Age')


# Sort descending

df.sort_values('Age', ascending=False)


# Sort by multiple columns

df.sort_values(['City', 'Age'], ascending=[True, False])


# Sort and place NaN first

df.sort_values('Salary', na_position='first')


# Sort by row index

df.sort_index() # ascending index

df.sort_index(ascending=False)
```

```
# Reset index after sort
df.sort_values('Age', ignore_index=True) # re-numbers 0,1,2...
```

SECTION 11

GroupBy & Aggregation

GroupBy follows the split-apply-combine pattern: split the DataFrame into groups based on a column, apply an aggregation function (sum, mean, count, min, max, etc.) to each group, then combine the results into a new DataFrame. agg() allows applying multiple or named aggregations at once. This is the Pandas equivalent of SQL's GROUP BY clause.

■ GroupBy Examples

```
# Mean salary per city
df.groupby('City')['Salary'].mean()

# Multiple aggregations on one column
df.groupby('City')['Salary'].agg(['min','max','mean','sum'])

# Different aggregations per column
df.groupby('City').agg({
'Age': 'mean',
'Salary': ['min','max']
})

# Named aggregation (clean output columns)
df.groupby('City')['Age'].agg(
avg_age='mean',
oldest='max',
youngest='min'
)

# Count rows per group
df.groupby('City').size()

# Iterate over groups
for name, group in df.groupby('City'):
```

```
print(name, group.shape)
```

SECTION 12

Merging & Joining

pd.merge() combines two DataFrames by matching values in one or more key columns — identical to SQL JOINs. 'inner' keeps only rows where the key exists in both DataFrames. 'left' keeps all rows from the left DataFrame (filling NaN for non-matching right rows). 'right' is the mirror of 'left'. 'outer' keeps all rows from both DataFrames. Use on= when the key column has the same name in both; use left_on / right_on when they differ.

■ Merge / Join Examples

```
employees = pd.DataFrame({'ID':[1,2,3], 'Name':['A','B','C']})
salaries = pd.DataFrame({'ID':[1,2,4], 'Salary':[50,60,70]})

# Inner join (only matching IDs: 1,2)
pd.merge(employees, salaries, on='ID')

# Left join (all employees, NaN if no salary)
pd.merge(employees, salaries, on='ID', how='left')

# Right join (all salary rows)
pd.merge(employees, salaries, on='ID', how='right')

# Outer join (all rows from both)
pd.merge(employees, salaries, on='ID', how='outer')

# Different key column names
pd.merge(df1, df2, left_on='emp_id', right_on='id')

# Merge on index
df1.join(df2.set_index('ID'), on='ID')
```

SECTION 13

Concatenating

pd.concat() stacks DataFrames either vertically (axis=0, adding more rows) or horizontally (axis=1, adding more columns). Setting ignore_index=True renumbers the row index from 0 after stacking. keys= adds an outer hierarchical index so you can identify the source. Use concat instead of repeatedly appending rows in a loop — it is far more efficient.

■ Concatenation Examples

```
df1 = pd.DataFrame({'A':[1,2], 'B':[3,4]})
df2 = pd.DataFrame({'A':[5,6], 'B':[7,8]})

# Stack rows (vertical) – default axis=0
result = pd.concat([df1, df2], ignore_index=True)
# A B
# 1 3
# 2 4
# 5 7
# 6 8

# Stack columns (horizontal)
pd.concat([df1, df2], axis=1)

# With source keys
pd.concat([df1, df2], keys=['source1','source2'])

# Align on columns (fills NaN for missing cols)
df3 = pd.DataFrame({'A':[9], 'C':[10]})
pd.concat([df1, df3], ignore_index=True) # B and C get NaN
```

SECTION 14

Apply, Map & ApplyMap

apply() is the most versatile transformation tool. On a Series it calls the function once per element. On a DataFrame with axis=1 it calls the function once per row. map() is element-wise and works only on a Series — ideal for replacing values using a dictionary or a simple function. applymap() (renamed to map() in newer Pandas) applies a function to every cell of a DataFrame. Use these instead of Python for-loops for speed.

■ Apply / Map Examples

```
# apply on Series – double every age
df['Age'].apply(lambda x: x * 2)
```

```
# apply with a named function
def grade(score):
return 'A' if score >= 90 else 'B' if score >= 75 else 'C'
df['Grade'] = df['Score'].apply(grade)

# apply row-wise (axis=1) – combine two columns
df['Summary'] = df.apply(
lambda row: f"{row['Name']} is {row['Age']} years old", axis=1
)

# map – replace city names with abbreviations
df['City'] = df['City'].map({'Delhi':'DL','Mumbai':'MB','Kolkata':'KL'})

# applymap / DataFrame.map – round all numeric cells
df[['Age','Salary']].map(lambda x: round(x, 2))
```

SECTION 15

String Operations

Pandas exposes the .str accessor on string Series, which lets you apply vectorised string methods to every element without writing a loop. These methods mirror Python's built-in str functions but operate on the entire column at once. They also handle NaN gracefully by returning NaN instead of raising an error. Common operations include case conversion, stripping whitespace, splitting, searching, and regex matching.

■ String Method Examples

```
# Case conversion
df['Name'].str.upper() # 'alice' → 'ALICE'
df['Name'].str.lower() # 'ALICE' → 'alice'
df['Name'].str.title() # 'alice bob' → 'Alice Bob'

# Whitespace
df['Name'].str.strip() # remove leading/trailing spaces
df['Name'].str.lstrip() # left strip only

# Length and character access
df['Name'].str.len() # number of characters
```

```
df['Name'].str[0] # first character


# Search
df['Name'].str.contains('li') # True/False mask
df['Name'].str.startswith('A')
df['Name'].str.endswith('e')


# Replace and split
df['Name'].str.replace('Bob','Robert')
df['FullName'].str.split(' ', expand=True) # split into cols


# Extract with regex
df['Phone'].str.extract(r'(\d{10})')
```

SECTION 16

DateTime Operations

Pandas has excellent datetime support through the pd.Timestamp type and the .dt accessor. Use pd.to_datetime() to convert string columns to proper datetime objects. Once converted, the .dt accessor unlocks year, month, day, hour, day-of-week, and many more attributes. Date arithmetic works naturally: subtracting two datetime columns returns a Timedelta. pd.date_range() creates a sequence of equally-spaced dates.

■ DateTime Examples

```
# Convert string column to datetime
df['Date'] = pd.to_datetime(df['Date'])
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y')


# Extract components via .dt accessor
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['Day'] = df['Date'].dt.day
df['Weekday'] = df['Date'].dt.day_name() # 'Monday'
df['Quarter'] = df['Date'].dt.quarter


# Date arithmetic
df['Days_Since'] = pd.Timestamp.today() - df['Date']
df['Days_Num'] = df['Days_Since'].dt.days
```

```
# Create a range of dates
dates = pd.date_range(start='2024-01-01', periods=7, freq='D')
months = pd.date_range('2024-01', periods=12, freq='MS')

# Filter by date
df[df['Date'] >= '2024-06-01']
df[df['Date'].dt.year == 2024]
```

SECTION 17

Pivot Tables & Reshaping

pivot_table() summarises data like an Excel pivot table — you specify which column provides row labels (index), which provides column labels (columns), which column to aggregate (values), and the aggregation function (aggfunc). melt() converts a wide DataFrame to a long (tidy) format, which is the format most machine-learning and plotting libraries prefer. stack() and unstack() move between levels of a hierarchical MultiIndex.

■ Pivot & Reshape Examples

```
# Pivot table – average salary by City and Senior status
df.pivot_table(
values='Salary',
index='City',
columns='Senior',
aggfunc='mean',
fill_value=0
)

# Multiple aggregations in pivot
df.pivot_table(values='Salary', index='City',
aggfunc=['mean','count'])

# melt – wide to long (tidy format)
pd.melt(df, id_vars=['Name'],
value_vars=['Math','Science'],
var_name='Subject', value_name='Score')

# stack / unstack
```

```
df.stack() # columns become rows

df.unstack() # innermost row level becomes columns


# crosstab – frequency table

pd.crosstab(df['City'], df['Senior'])
```

SECTION 18

Value Counts & Unique

value_counts() is one of the most used exploratory functions — it returns a Series showing how many times each unique value appears, sorted by frequency. unique() returns an array of distinct values (in order of appearance). nunique() returns the count of distinct values. normalize=True in value_counts() returns proportions (0–1) instead of raw counts.

■ Value Counts & Unique Examples

```
# Frequency of each value

df['City'].value_counts()

# Mumbai 5

# Delhi 3

# Kolkata 2


# As proportions (0.0 – 1.0)

df['City'].value_counts(normalize=True)


# Include NaN in count

df['City'].value_counts(dropna=False)


# Array of unique values

df['City'].unique() # array(['Mumbai','Delhi','Kolkata'])


# Count of unique values

df['City'].nunique() # 3


# Unique pairs across two columns

df[['City','Senior']].drop_duplicates()
```

SECTION 19

Index Operations

The index is the row label of a DataFrame or Series. By default it is a RangeIndex (0, 1, 2, …). set_index() promotes any column to become the new index — this enables fast label-based lookups. reset_index() reverses this, converting the index back into a regular column. reindex() lets you specify a new index order, inserting NaN for any labels not found in the original data.

■ Index Operation Examples

```
# Set a column as the index

df.set_index('Name', inplace=True)

df.loc['Alice'] # now works by name!


# Reset back to 0,1,2...

df.reset_index(inplace=True) # 'Name' becomes a column again


# Rename the index

df.index.name = 'RowID'


# Reorder / reindex rows

df.reindex([2, 0, 1])


# Reindex with a new set of labels (NaN for missing)

df.reindex(range(10)) # rows 3-9 will be NaN


# MultiIndex (hierarchical index)

df.set_index(['City','Name'], inplace=True)

df.loc[('Mumbai','Alice')]
```

SECTION 20

Statistical Functions

Pandas provides a comprehensive set of descriptive statistics methods that operate column-wise by default (axis=0). They skip NaN by default (skipna=True). cumsum() and cumprod() return running totals. pct_change() computes the percentage change between consecutive rows, useful for time-series. corr() computes the Pearson correlation coefficient — a value near 1 means strong positive correlation, near -1 means strong negative.

■ Statistical Function Examples

```
# Basic descriptive statistics
```

```
df['Age'].mean() # arithmetic mean

df['Age'].median() # middle value

df['Age'].mode()[0] # most frequent value

df['Age'].std() # standard deviation

df['Age'].var() # variance

df['Age'].min() # minimum

df['Age'].max() # maximum

df['Age'].sum() # total


# Range and quantiles

df['Age'].quantile(0.25) # 25th percentile

df['Age'].quantile(0.75) # 75th percentile


# Cumulative stats

df['Sales'].cumsum() # running total

df['Sales'].cumprod() # running product

df['Sales'].pct_change() # % change row-over-row


# Correlation

df['Age'].corr(df['Salary']) # single pair

df.corr() # full correlation matrix

df.cov() # covariance matrix
```

SECTION 21

Conditional Selection (where / mask / query)

where() keeps values where the condition is True and replaces the rest with NaN (or a given value). mask() is the inverse — it replaces values where the condition is True. query() lets you filter rows using a concise SQL-like string expression, which is often more readable than bracket notation for complex filters. copy() creates an independent deep copy so edits do not affect the original DataFrame.

■ Conditional Selection Examples

```
# where – keep values where condition is True, else NaN

df['Age'].where(df['Age'] > 25)


# where with a replacement value instead of NaN

df['Age'].where(df['Age'] > 25, other=0)
```

```
# mask – replace values where condition is True
df['Age'].mask(df['Age'] > 50, other=50) # cap at 50

# query – SQL-like filtering (very readable)
df.query('Age > 25 and City == "Mumbai"')
df.query('Salary >= 60000')

# np.where – element-wise ternary
import numpy as np
df['Tag'] = np.where(df['Age'] >= 30, 'Senior', 'Junior')

# Deep copy (edits won't affect original)
df_copy = df.copy()
```

SECTION 22

Quick Reference Cheat Sheet

Task	Syntax	Notes
Install	`pip install pandas`	Run once in terminal
Import	`import pandas as pd`	Standard alias
Read CSV	`pd.read_csv('f.csv')`	index_col, sep, dtype params
Write CSV	`df.to_csv('f.csv',index=False)`	index=False avoids extra col
Shape	`df.shape`	(rows, cols) tuple
First rows	`df.head(n)`	Default n=5
Stats summary	`df.describe()`	count,mean,std,min,max
Select col	`df['col']`	Returns Series
Multi cols	`df[['a','b']]`	Returns DataFrame
By label	`df.loc[row, col]`	End label inclusive
By position	`df.iloc[r, c]`	End position exclusive
Filter rows	`df[df['col'] > val]`	Boolean mask
Add column	`df['new'] = values`	Or use insert()
Drop column	`df.drop('col', axis=1)`	axis=1 for columns
Fill NaN	`df.fillna(value)`	Or method='ffill'
Count NaN	`df.isnull().sum()`	Per column

Task	Syntax	Notes
Sort	`df.sort_values('col')`	ascending=False for desc
Group & agg	`df.groupby('col').mean()`	agg() for multiple fns
Merge	`pd.merge(df1,df2,on='col')`	how='left/right/outer'
Concat rows	`pd.concat([df1,df2])`	ignore_index=True
Apply fn	`df['col'].apply(fn)`	lambda ok
String ops	`df['col'].str.upper()`	.str accessor
To datetime	`pd.to_datetime(df['col'])`	Then use .dt accessor
Pivot table	`df.pivot_table(...)`	Like Excel pivot
Value counts	`df['col'].value_counts()`	normalize=True for %
Correlation	`df.corr()`	Full matrix
Query	`df.query('Age > 25')`	SQL-like syntax
Copy	`df.copy()`	Independent deep copy

Pandas version used: 2.x | Python 3.8+ | Study Notes – All Sections Covered